Review 2: CDFG, FSM and Task Schedule

Gesture-Controlled Sentry Turret · critical-function CDFG, system FSM and schedulability analysis

Project	Gesture-Controlled Sentry Turret
Scope tier	Core tier: glove-to-turret BLE control, rule-based gestures, fail-safe wheel task, web control
Review date	01 / 10 / 2026
Review 1 result	Approved with conditions (reviewer: Nimal J Kumar); items in §1.1

Member	Reg. no.	Module
Sankepalli Kharanshu	25BCE2093	Turret mechanical build
Tom Biju	25BCE0973	Turret firmware
Ahmed Sahil	25BCE0252	Glove and gesture recognition
Harish Saravanan	25BCE0987	Integration, tests, demo

1 Recap

We present the system as a tracking and pointing platform. A glove with an IMU is worn on the hand. An ESP32 on the glove classifies the motion and sends a short command over BLE; a second ESP32 on the turret moves a pan-tilt servo mount, switches a laser, and reads an ultrasonic range sensor and a PIR sensor. The turret also shows state on an OLED, an RGB ring and a buzzer. The design is a spec-level model: it describes what we intend to build and the numbers are estimates from datasheets and instruction counts, not measurements.

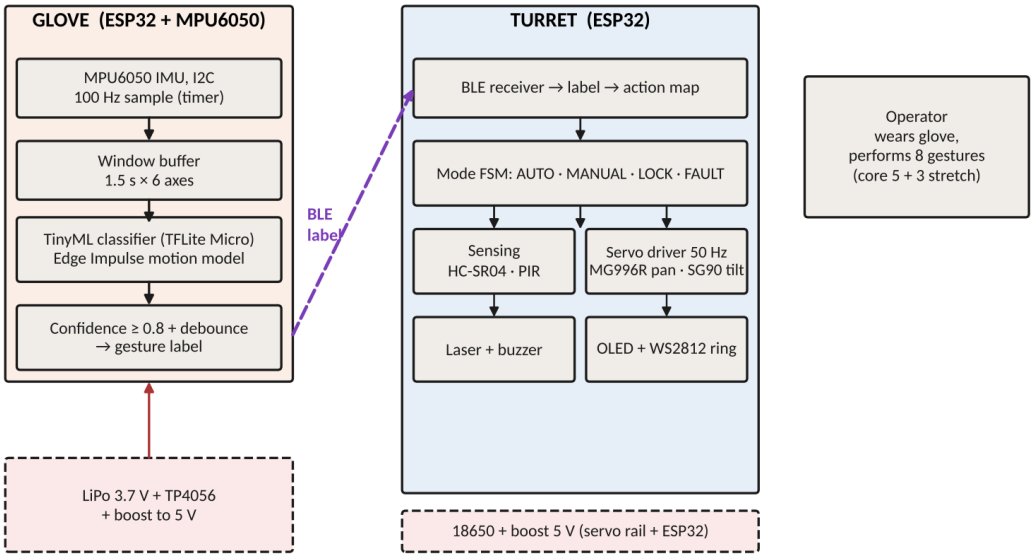


Figure 1 — System block diagram at Review 2.

1.1 Conditions from Review 1

Review 1 item	How this review addresses it
Condition: reduce the baseline gesture set to four; other four optional	Baseline is four classes; the model output has eight only if the stretch classes are added (§2.3 m5, §6 plan)
Condition: drop the projectile / "sentry" framing	Described as a tracking and pointing platform; the laser is a visible pointer module only (§1)
Condition: supply Tom Biju's email	Completed: Tom Biju's email has been supplied
Mandatory (Review 2): rule-based gesture fallback	§6 plan: threshold rules run alongside the model, and the demo falls back to them if the model underperforms
Mandatory (Review 2): confusion matrix on held-out data, measured latency and BLE range	§5 and §6: held-out recordings and timing captures are scheduled tests
Mandatory (Review 2): safe state: on BLE loss the turret centres, stops and says so on the OLED	§3.4: FAULT centres the servos, turns the laser off and shows LINK LOST

Source: Review 1 sheet, proposal review 05 of 19. The first two mandatory additions were due at Review 1 and are not repeated here.

2 CDFG of the critical function

2.1 Candidate functions

Candidate	Rough WCET	Deadline	Why it matters	Chosen?
`classify_window` (glove): DSP + neural net on a 1.5 s IMU window	≈ 24 ms	200 ms (window hop)	Longest compute task; decides every gesture	**Yes**
`imu_sample`: read MPU over I2C	1.6 ms	10 ms	Short, fixed cost; no data-dependent path	No
`oled_refresh`: 1 KB frame over I2C	28 ms	250 ms	Long but display only; no safety effect	No
`ultrasonic`: trigger, wait for echo	12 ms	60 ms	Blocking, but a single wait with no branches	No

We chose `classify_window`: it has the most computation, data-dependent branches (confidence and debounce) and its latency adds directly to the glove-to-turret delay.

2.2 CDFG

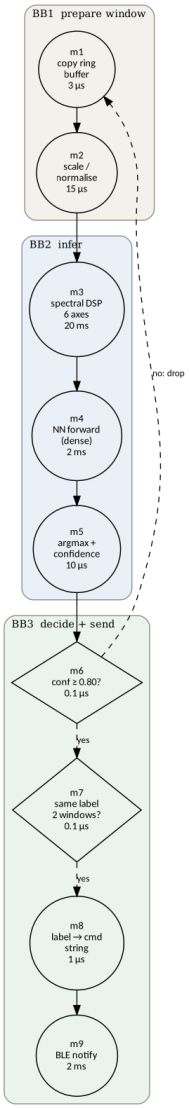


Figure 2 — CDFG of `classify_window`*. Circles = operations, diamonds = decisions, dashed = control flow; costs are estimates.*

2.3 Node table

Node	Operation	Basic block	Computation	Predecessor	Successor	WCET (est.)
m1	copy ring buffer	BB1	memcpy 1.5 s × 6 axes	—	—	3 µs
m2	scale / normalise	BB1	float multiply per sample	m1	m3	15 µs
m3	spectral DSP, 6 axes	BB2	FFT-based features, 6 axes	m2	m4	20 ms
m4	NN forward (dense)	BB2	int8/float dense layers	m3	m5	2 ms
m5	argmax + confidence	BB2	compare 4 baseline (8 with stretch) outputs	m4	m6	10 µs
m6	confidence ≥ 0.80?	BB3	compare, branch	m5	m7 / m1	0.1 µs
m7	same label 2 windows?	BB3	compare with previous label	m6	m8	0.1 µs
m8	label → command string	BB3	table lookup	m7	m9	1 µs
m9	BLE notify	BB3	stack call, 20-byte payload	m8	—	2 ms

Costs are estimates: m3 and m4 from the operation counts of the planned model at 240 MHz, m9 from a typical BLE notify call, the rest from instruction counts. No measurement exists at this stage.

2.4 Critical path

m1 → m2 → m3 → m4 → m5 → m6 → m7 → m8 → m9 = 3 + 15 + 20,000 + 2,000 + 10 + 0.1 + 0.1 + 1 + 2,000 = **24,029 µs ≈ 24.0 ms**, against a 200 ms window hop (slack ≈ 176 ms). The spectral step m3 is 83% of it and is the first thing to optimise if the deadline is ever missed (fewer axes, smaller FFT). If m6 fails the confidence test, control returns to m1 and nothing is sent, so the false-gesture path costs no radio time.

3 Finite-state machine

3.1 Type

Mealy. Outputs depend on the state and the input event: the same gesture `g_lock` fires the laser from TRACKING and from MANUAL, and `g_mode` goes to a different state in each case. Outputs are written as `event / action` on the edges.

3.2 Diagram

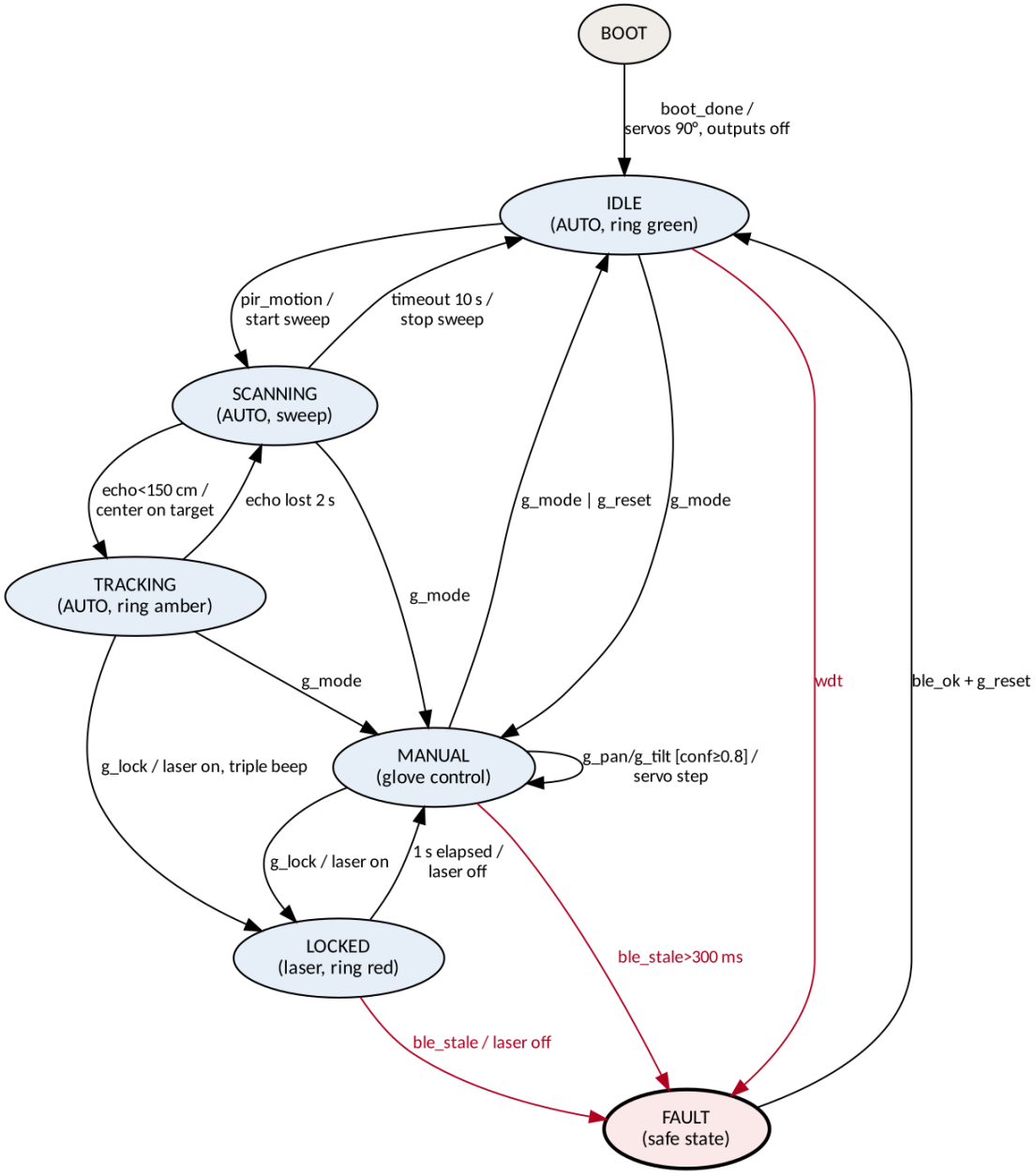


Figure 3 — System FSM. Red edges lead to FAULT, the safe state.

3.3 Transition table

Current state	Event / guard	Next state	Output / action
BOOT	boot_done	IDLE	servos to 90°, all outputs off
IDLE	pir_motion	SCANNING	start 10 s servo sweep
IDLE	g_mode	MANUAL	glove takes over
IDLE	wdt	FAULT	all outputs off
SCANNING	echo < 150 cm	TRACKING	centre servo on the target
SCANNING	timeout 10 s	IDLE	stop sweep
SCANNING	g_mode	MANUAL	stop sweep
TRACKING	echo lost 2 s	SCANNING	resume sweep
TRACKING	g_lock	LOCKED	laser on, triple beep
TRACKING	g_mode	MANUAL	—
MANUAL	g_pan / g_tilt, conf ≥ 0.8	MANUAL	servo step
MANUAL	g_lock	LOCKED	laser on
MANUAL	g_mode or g_reset	IDLE	—
MANUAL	ble_stale > 300 ms	FAULT	laser off, servos centre, OLED LINK LOST
LOCKED	1 s elapsed	MANUAL	laser off
LOCKED	ble_stale	FAULT	laser off, servos centre, OLED LINK LOST
FAULT	ble_ok and g_reset	IDLE	outputs stay off until IDLE

3.4 Fault and safe state

The safe state is **FAULT**: laser low, buzzer off, servos centre to 90° and stop, the OLED shows LINK LOST and the RGB ring is red, as Review 1 requires. It is entered when no BLE packet has arrived for more than 300 ms (while the glove is in control) or when the watchdog timer fires. Leaving FAULT needs both a fresh link and an explicit `g_reset`, so a reconnecting glove cannot move the turret by surprise. Because LOCKED self-expires after 1 s, the laser cannot stay on without a fresh command.

3.5 Completeness matrix

✓ = transition defined. – = event ignored by design (stays in state, no output). Events: pir = pir_motion, echo = echo<150, g_mode, g_lock, g_reset, stale = ble_stale, wdt.

State	pir	echo	g_mode	g_lock	g_reset	stale	wdt
IDLE	SCANNING	–	MANUAL	–	–	FAULT	FAULT
SCANNING	–	TRACKING	MANUAL	–	–	FAULT	FAULT
TRACKING	–	–	MANUAL	LOCKED	–	FAULT	FAULT
MANUAL	–	–	IDLE	LOCKED	IDLE	FAULT	FAULT
LOCKED	–	–	–	–	–	FAULT	FAULT
FAULT	–	–	–	–	IDLE*	–	–

*Every state has a defined response to stale and wdt, so the safe state is reachable from everywhere. *FAULT → IDLE also requires ble_ok. LOCKED leaves only by its 1 s timer; the timer edge is the only way the laser turns off in normal use.*

4 Task schedule

4.1 Task set

Seven tasks, rate-monotonic priorities (shortest period highest). Deadline = period. Blocking by shared resources is neglected here and discussed in §4.5.

Task	Period T (ms)	WCET C (ms)	Deadline D (ms)	Utilisation C/T
imu_sample	10	1.6	10	0.16
servo_update	20	0.05	20	0.0025
fsm_ring	50	0.5	50	0.01
ultrasonic	60	12	60	0.2
classify_window	200	24	200	0.12
ble_notify	200	2	200	0.01
oled_refresh	250	28	250	0.112

4.2 Approach

Rate-monotonic scheduling on FreeRTOS, one task per row, priorities by period. Rate-monotonic is optimal for fixed priorities with deadlines equal to periods, and FreeRTOS provides preemptive fixed-priority scheduling directly. The alternative, a cyclic super-loop, was rejected because the 24 ms classification and 28 ms display transfer would delay the 10 ms IMU read by more than its period.

4.3 Schedulability analysis

Total utilisation U = 0.6145. The Liu–Layland bound for n = 7 is 7(2^(1/7) − 1) = 0.7286, and 0.6145 < 0.7286, so the set is schedulable by the sufficient test. To confirm, we compute response times with the recurrence R = C + Σ ⌈R/T_j⌉ C_j over higher-priority tasks:

Task	C (ms)	D = T (ms)	Response time R (ms)	R ≤ D?
imu_sample	1.6	10	1.60	yes
servo_update	0.05	20	1.65	yes
fsm_ring	0.5	50	2.15	yes
ultrasonic	12	60	15.75	yes
classify_window	24	200	44.65	yes
ble_notify	2	200	46.65	yes
oled_refresh	28	250	95.25	yes

Worked example, classify_window: R0 = 24; interference from imu (1.6 × ⌈24/10⌉ = 4.8), servo (0.05 × 2 = 0.1), fsm (0.5 × 1 = 0.5), ultrasonic (12 × 1 = 12) → 41.4; iterating, 41.4 gives 1.6 × 5 + 0.05 × 3 + 0.5 + 12 = 20.65 + 24 = 44.65, which then stays at 44.65. All response times are below their deadlines; the worst slack ratio is oled_refresh at 95.25 of 250 ms.

4.4 Timeline

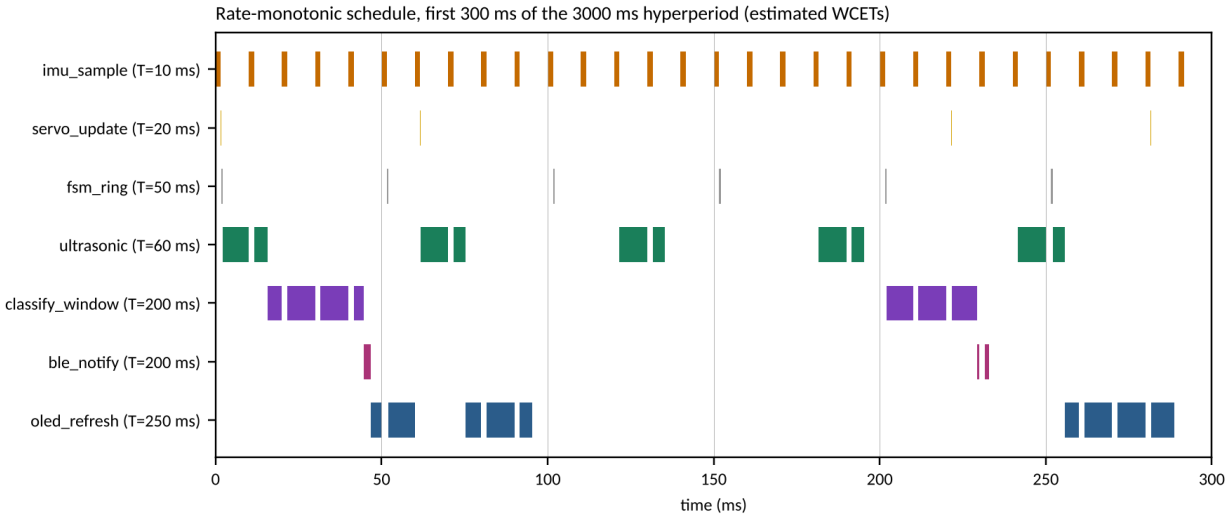


Figure 4 — Timeline of the task set over 250 ms (worst-case WCET, all tasks released together).

4.5 Shared resources

Resource	Used by	Protection	Blocking bound
I2C bus (turret: OLED)	oled_refresh only	No sharing; the glove IMU bus is on the other board	0
Servo PWM state	servo_update, fsm_ring	Mutex with priority inheritance	≤ 0.05 ms (servo write)
Gesture ring buffer (glove)	imu_sample (producer), classify_window (consumer)	Double buffer; classify copies in m1 (3 µs critical section)	≤ 0.003 ms
Last-gesture variable	classify_window, ble_notify	Mutex	≤ 0.002 ms

Priority inheritance bounds blocking to the longest critical section (0.05 ms), which is far below the slack of every task, so including it does not change the result of §4.3.

5 Planned traceability

Model element	Planned file	Planned function	Test that checks it
CDFG m1–m2	glove/window.h	`prepare_window()`	Host unit test: window length and scaling
CDFG m3–m5	glove/classify.h	`classify_window()`	Host test on recorded gestures; WCET by GPIO toggle
CDFG m6–m8	glove/classify.h	`debounce_and_encode()`	Host test: confidence threshold, two-window rule
CDFG m9	glove/glove.ino	`send_command()`	Serial log of sent packets
FSM states	turret/fsm.h	`fsm_step(event)`	Table-driven test of every row in §3.3
FSM FAULT	turret/fsm.h	`fault_check()`	Fault injection: unplug glove, stall a task
Task set	turret/turret.ino	`xTaskCreate` × 6	Timing capture of each task start and end

6 Plan to Final

Week	Work	Owner	Risk and fallback
1	IMU driver, ring buffer, data collection for 4 baseline gestures	Ahmed Sahil	Too little data → keep four crisp gestures; stretch classes stay optional
2	Classifier training and on-device port; BLE link	Ahmed Sahil, Harish Saravanan	NN too slow → threshold-based rules as fallback
2	Servo, OLED, ring, buzzer, FSM on turret	Tom Biju	RTOS bugs → single super-loop fallback
3	Mount pan-tilt, laser, sensors; wiring	Sankepalli Kharanshu	Bracket mismatch → printed or cut adapter
3	Integration; timing captures for §4	Harish Saravanan	Missed deadline → shrink DSP (m3)
4	Final tests, report, demo	All	—

7 Contribution

Member	Contribution to this review	Share %
Sankepalli Kharanshu	Hardware inventory, mounting plan, power notes	25
Tom Biju	FSM, transition table, completeness matrix	25
Ahmed Sahil	CDFG, node table, critical path	25
Harish Saravanan	Schedule analysis, traceability, plan to Final	25

References

[1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, 1973.
[2] Espressif Systems, *ESP32 Series Datasheet*; *Arduino-ESP32 BLE API*.
[3] InvenSense, *MPU-6500 Product Specification*.
[4] FreeRTOS, *Reference manual: scheduling and mutexes with priority inheritance*, freertos.org.

Submission checklist

Item	Done
CDFG with critical path and node table	✓
FSM with transition table and safe state	✓
Schedulability analysis with response times	✓
Review 1 conditions listed	✓
Diagrams explainable by every member	Check before presenting